

Phase 1 validation report

Contents

Verdict

1. The agents, on real data
2. Efficiency
3. Product features
4. Published results, re-run
5. Defects this validation found — all fixed
6. What this validation does not show
7. Reproduce it

A rigorous check of every Phase 1 agent and main feature: does each one do exactly what it claims, on real data, repeatably, and how fast? Everything here is measured by committed scripts, with no LLM key (as the product ships), on 21–22 September 2026. Nothing is estimated; where something was not re-measured, it says so.

Verdict

Area	What was checked	Result
Agent pipeline	All 987 real sprints through the live analysis endpoint, at sprint end and halfway: 2,961 analyses	Every check passed — 0 errors, 0 invalid outputs, same answer every time
Planning, Progress, Workload agents	Every figure each reports, against an independent recount from the raw sprint	1,974 / 1,974 match, per agent
Risk agent	Scores vs. direct graph computation; every cited node real; delay predictions vs. what each sprint really did	1,974 / 1,974 match · 40,001 / 40,001 citations real · reproduces the published figures exactly
Recommendation agent	Every suggested reassignment applied and the risk recomputed	Predicted effect held in 1,524 / 1,524 cases; severity improved in 62%
Published evaluations	Every one re-run from scratch	All reproduce — after fixing an evaluation clock that had silently broken one of them
Roles	Every protected endpoint × every role, read off the code	34 × 5 = 170 checks pass — after fixing five security defects this validation found
Features	Test suites, the demo replay, response times	82 + 85 + 58 tests pass; 14 / 14 demo claims; every feature responds in under a second
Load	Up to 50 people at once, 5 analyses at once, 25 simultaneous sign-ins	0 failed requests — after fixing three defects the load test found (7 failures before)

1. The agents, on real data

Method (ai-service/eval/validate_phase1.py). Each of the 987 eligible TAWOS sprints (31 projects; median 23 tasks, largest 119) is sent to POST /api/v1/analyze — the endpoint and scope the backend uses when someone clicks **Run Analysis** — in-process, with no LLM key. Each sprint is analysed as it stood at its end and again halfway through, and the end-of-sprint analysis is repeated to test determinism: **2,961 analyses**. Historical sprints are moved in time so the moment analysed is "now", exactly as the evaluation scripts pin their clocks.

Coordinator

Check	Result
Analysis completes	1,974 / 1,974
Returns all five specialist outputs	1,974 / 1,974
Same sprint, analysed twice, gives an identical result	987 / 987
Response time, in process (all 1,974)	median 23 ms, 95th percentile 31 ms (48 / 70 ms in the first run: timings vary with the laptop's load)
... for sprints under 25 tasks / 25–49 / 50–99 / 100+	22 / 24 / 29 / 37 ms median

Planning, Progress and Workload

Each agent's output is compared with a recount done independently from the raw sprint data, not with the agent's own code — so a wrong input (the kind of bug that once made every specialist fail) would show up as a mismatch.

Agent	Check	Result
Planning	Sprint count, task count, readiness, capacity, largest plan and risk level	1,974 / 1,974
Progress	Completion rate	1,974 / 1,974
Progress	The blocked tasks it lists as stalled	1,974 / 1,974
Workload	Team size, and exactly who is overloaded and under-used	1,974 / 1,974
All three	Output valid against its schema; no error output	1,974 / 1,974 each
All three	LLM calls that reached the network without a key	0 of 8,883 attempts — every one refused before any I/O

Their own verdicts, against the real outcome. Each specialist also gives a risk level, shown to users beside the Risk agent's. It does not feed the product's risk score, but it is scored here the same way (a warning = anything above *low*; the 270 held-out sprints):

Agent	Halfway: warned · precision · recall · F1	Sprint end: warned · precision · recall · F1
Risk (the product's verdict)	205 · 0.581 · 0.902 · 0.706	239 · 0.552 · 1.000 · 0.712
Progress	208 · 0.582 · 0.917 · 0.712	81 · 1.000 · 0.614 · 0.761
Workload	144 · 0.556 · 0.606 · 0.580	146 · 0.555 · 0.614 · 0.583
Planning	160 · 0.456 · 0.553 · 0.500	166 · 0.458 · 0.576 · 0.510

- **The Planning agent now warns, and its warning is a weak lateness signal.** It warns when a sprint plans more story points than the team can finish: the capacity a manager sets in project settings, or else the average the team completed in its last three finished sprints (a sprint with none before it gets no warning). Until this validation the backend supplied no capacity, so the agent never warned at all. Its capacity and plan figures match the independent recount in 1,974 / 1,974 runs. On real data it is not a good predictor of a late sprint: teams in TAWOS routinely plan more than they finish (80% of held-out sprints with a known capacity are over it), so the plan-to-capacity ratio ranks late sprints above on-time ones only a little better than chance (AUC **0.60** halfway, **0.62** at sprint end, held-out) and its F1 is below simply flagging every sprint (0.66). Its threshold was fixed in advance, not tuned on these sprints. It is a planning check, not a delay predictor; the Risk agent's pace signal (AUC 0.79) remains that.
- **Progress matches the Risk agent halfway through** (0.712 vs 0.706): a plain completion-rate rule does as well as the pace projection. Its perfect sprint-end precision is built in — a sprint less than half done is late by definition.
- **Workload is a weak lateness signal**, as expected: it measures uneven load, not delay.

Risk

Check	Result
Scores and overall level equal a direct GraphMetrics computation	1,974 / 1,974
Every finding cites the evidence it rests on	7,102 / 7,102 findings
Every cited task / person / component is a real node of that sprint's graph	40,001 / 40,001
Every other evidence reference resolves	7,102 / 7,102

Delay prediction — the system's delay verdict (anything but *low*) against whether the sprint really finished late ($\geq 30\%$ of its issues unresolved at the end):

When	Projects	Sprints	Precision	Recall	F1	Accuracy
Sprint end	all	987	0.630	1.000	0.773	0.655
Sprint end	held-out	270	0.552	1.000	0.712	0.604
Halfway	all	987	0.675	0.957	0.792	0.704
Halfway	held-out	270	0.581	0.902	0.706	0.633

These are produced through the live API. For all 987 sprints, at both moments, every cell of the confusion matrix is **identical** to the published evaluation scripts' results, and the held-out figures equal their published values — so the product computes what

the paper reports. Sprint-end recall of 1.00 is structural (the label and the rule test the same condition after the deadline); the halfway figures are the predictive ones. From the published evaluation, re-run and unchanged: held-out AUC **0.792** halfway and **0.874** at three-quarters; F1 +0.050 and +0.099 over flagging every sprint (95% CI [+0.006, +0.094] and [+0.049, +0.148], McNemar $p < 0.001$); held-out sprint-end F1 95% CI [0.610, 0.814]. A simple open-work-share baseline ranks as well (AUC 0.802 / 0.881) — the projection's value is the early, evidence-carrying warning, not better ranking.

Synthetic correctness (180 generated projects with a known planted problem): precision, recall and F1 **1.00 for all six risk types**, and each detector changes verdict exactly at its documented threshold (overdue ratio between 0.08 and 0.10; silent ratio between 0.2 and 0.3; workload skew across 1.50).

Recommendation

Check	Result
Every recommendation is grounded in an actual finding	4,363 / 4,363
Every risk at medium or worse gets a recommendation	4,363 / 4,363
Output valid; no error output	1,974 / 1,974

Coverage by risk type: delay 1,743, workload 1,808, silent members 779, dependency 33. **Knowledge concentration and coordination never reached medium on any real sprint** — the known limit that those signals rarely appear inside one sprint's window.

Suggested reassignments, applied. For every workload recommendation, each suggested move is applied to the sprint and the risk recomputed:

Check	Result
Workload recommendations	1,808 — 1,524 (84%) come with moves; in 284 no move could help
The "before" figure matches the board	1,524 / 1,524
Every move is valid (the task belongs to the person it moves from; the receiver is on the team)	1,524 / 1,524
No single move raises the heaviest load	1,524 / 1,524
Predicted heaviest load after the moves = actual	1,524 / 1,524
Predicted severity after the moves = actual	1,524 / 1,524
Severity actually moves down at least one level	943 / 1,524 (62%)
Reduction in the heaviest load	median 6 points, mean 9.5

Of the 581 that did not change level, 519 were *critical* and stayed *critical*: teams so uneven that the planner's three moves cannot bring them down a band. The prediction still told the user so in advance. This validates the fix inside the model; whether real teams would follow it is not measured.

The language-model layer (optional)

Check	Result	Re-measured?
Fabricated citations stripped before reaching the user	200 / 200	yes
No call without a real key (placeholder included)	0 of 8,883 attempts	yes
Agreement between the model's verdict and the deterministic one	Cohen's κ 0.95 (n = 37)	no — needs a paid key
Output validity	41.1% end-to-end; 54–95% for the model itself	no — needs a paid key
Graph pipeline vs. sending the whole project	F1 1.000 vs 0.947 (n = 36, not significant)	no — needs a paid key

2. Efficiency

Measure	Result
Prompt growth while the project grows 154× (13 → 2,003 tasks)	1.47× (266 → 390 tokens)
Naive whole-project prompt at 2,003 tasks	200,916 tokens → 515× more than the graph prompt
Graph build + all six risks, 203-task project	median 4.7 ms

The naive prompt is larger than when first published (189,834 tokens, 487×) because tasks and sprints gained two fields since; the graph prompt is unchanged. The published 487× is the conservative figure.

3. Product features

Tests. Backend **82** (SQLite locally; SQLite and PostgreSQL 16 in CI), AI service **85**, frontend **58** plus a clean type check. The demo replay (check_demo) holds **14 / 14** claims.

Roles (test_permission_matrix.py). The permission each endpoint requires is read off the code, then every protected endpoint (34 route/method pairs) is called as each of the five roles: **170 / 170** behave exactly as the policy says. A second check fails the build if any data-changing endpoint is added without a role check.

Code coverage — share of statements the test suites execute, excluding the tests themselves (pytest-cov; the backend measured with greenlet tracing, without which SQLAlchemy's async layer hides code after the first database call and the figure reads 7 points too low):

Area	Covered
Backend code the MVP uses (API routes, models, schemas, services)	74%
... of which task / project / auth routes	85% / 84% / 89%
Backend code outside the MVP (meetings, admin, background jobs)	18%
AI service, all	81%
... of which the knowledge graph and metrics	93%
Frontend, all source files (npm run test:coverage)	24%
... analysis panel / hooks / API client and utilities / project settings page	89% / 89% / 81% / 89%
... other pages, task board components, team panel	0–26%: exercised by the demo replay and by hand, not by unit tests

Response times — end to end over HTTP against the running stack with the real Mesos sprint (30 tasks, 13 people), SQLite, no LLM key, one Windows laptop; 15 timed requests each after 2 warm-ups:

Feature	Median	95th percentile
Sign in	390 ms	400 ms
Workspace	10 ms	11 ms
Project	8 ms	10 ms
Task board (30 tasks)	14 ms	15 ms
Dependencies / sprints / team	10 / 11 / 14 ms	12 / 14 / 17 ms
Dashboard	12 ms	16 ms
Task discussion	11 ms	13 ms
Create / edit / move / delete a task	25 / 22 / 22 / 23 ms	29 / 25 / 25 / 26 ms
Run analysis (backend + AI service)	715 ms	754 ms
Analysis history	24 ms	42 ms

Sign-in is slow by design: password hashing (bcrypt) is deliberately expensive. Of the analysis time, the AI pipeline itself takes under 50 ms; the rest is the backend loading the project, calling the AI service and saving the result.

Many people at once (`load_test.py`, quick levels; same laptop, SQLite, one server process per service, the load generator on the same machine, so these are a floor). Each virtual user repeats a session mix of the board, dashboard, project, team, sprints, dependencies, a discussion, and a create-move-delete on a task, with no pause between requests:

Load	Before the fixes below	After
1 user	57 req/s · median 15 ms	57 req/s · median 15 ms
10 users	59 req/s · p95 345 ms	77 req/s · p95 212 ms
25 users	70 req/s · p99 2.0 s	74 req/s · p99 0.7 s
50 users	60 req/s · p99 7.1 s · 3 failed	78 req/s · p99 1.4 s · 0 failed
5 analyses at once	median 1.9 s	median 2.2 s
25 people signing in at once	all done in 9.8 s · 4 failed	all done in 1.9 s · 0 failed

The server peaks near 75 requests a second on SQLite; beyond that more users means longer waits, not failures. With a person making a request every few seconds, that is on the order of a few hundred people working at once on this laptop. CI now runs the same quick test on SQLite and PostgreSQL on every push and fails on any failed request.

4. Published results, re-run

Evaluation	Re-run result
TAWOS, all 987 sprints	identical to the published file
TAWOS, 270 held-out sprints	identical
Held-out bootstrap confidence interval	identical
Mid-sprint checkpoints and paired tests	identical
Synthetic detection	had broken — F1 1.00 → 0.92; identical again after the fix below
Threshold boundaries	exact again after the same fix
Token scaling	graph side identical; naive baseline larger (see §2)

5. Defects this validation found — all fixed

Defect	Impact	Fix
The admin deactivation endpoint had no role check	Any signed-in member, even a viewer, could deactivate any account — the owner's included	Admin area limited to admins and owners
Member removal and deactivation had no rank check	An admin could remove or lock out the owner	Nobody acts on someone above them, or on themselves
Member removal always crashed	It set a required field to empty	Removal now revokes roles and project memberships and disables the account
A disabled account kept working until its token expired	Up to 30 minutes of continued changes	Every permission check refuses a disabled account
Meeting and organizational-memory endpoints had no role check	A viewer could create and change data	Checked against the permissions the policy already defined
The admin statistics endpoint always crashed	Used a name before importing it	Imports fixed
The synthetic evaluation used the wall clock	A month after publication, healthy scenarios' due dates had passed and read as overdue: F1 silently fell to 0.92	Scenarios judged at their own fixed clock; the result is now a CI test

None of these was in a path the web app uses, which is why no test or demo had exercised them; all were reachable through the deployed API. Each now has a regression test.

The load test found three more, all fixed:

Defect	Impact	Fix
Password hashing ran on the server's event loop	Every sign-in (~0.3 s of bcrypt) froze all other requests; 25 at once took 9.8 s and 4 failed	Hashing runs on a worker thread
Every SQL statement was logged whenever DEBUG was on — the default, Docker Compose included	Slower requests everywhere	Statement logging is its own setting, SQL_ECHO, off by default
SQLite gave up on a busy write lock after 5 s	Under 50 users, some task writes failed with "database is locked"	Write-ahead logging, and a 30 s wait for the lock

6. What this validation does not show

- **Whether the explanations help people** — there is no human evaluation yet. The study is ready to run: protocol, forms, two real sprints, setup script, answer key and analysis script are in docs/user-study/; it needs 8–12 participants.
- **Knowledge and coordination risk on real data** — they never reach *medium* on a real sprint.
- **The language-model layer's current behaviour** — its three results above are from the published runs; re-measuring needs a paid key.
- **Whether real teams would follow the suggested moves** — only that the moves do what they predict.
- **Production-scale performance** — the load test ran on one laptop on SQLite; CI repeats it on PostgreSQL on a shared runner, but nothing has been measured on production hardware.

The charts and a per-agent completeness matrix are in PHASE1_PERFORMANCE_METRICS.pdf.

7. Reproduce it

From ai-service/ (needs the local TAWOS database for the second and later lines):

```
python -m eval.run_eval
python -m eval.boundary_eval
python -m eval.validate_phase1
python -m eval.datasets.tawos_holdout_eval
python -m eval.datasets.tawos_midsprint_eval
```

From backend/, with the backend and AI service running:

```
python -m app.scripts.check_demo
python -m app.scripts.measure_latency --runs 15
python -m app.scripts.load_test --quick
```

Results: ai-service/eval/phase1_validation_result.json, ai-service/eval/phase1_api_latency_result.json,
ai-service/eval/phase1_load_test_result.json.